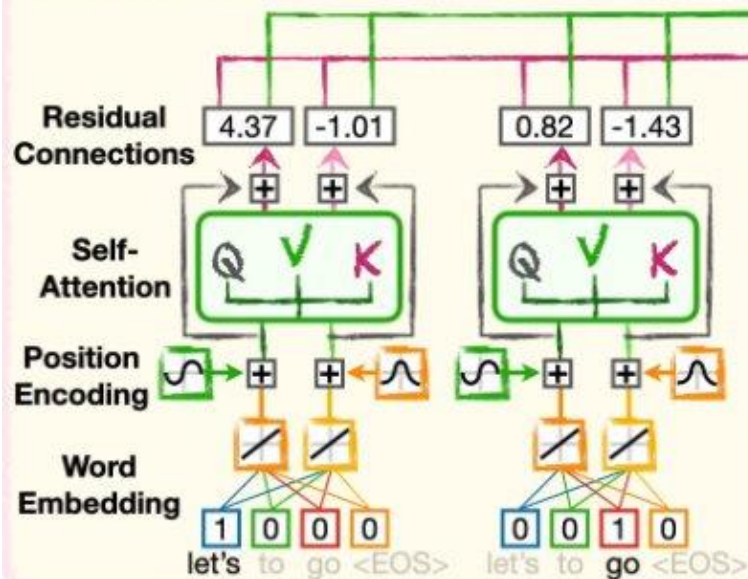


Background

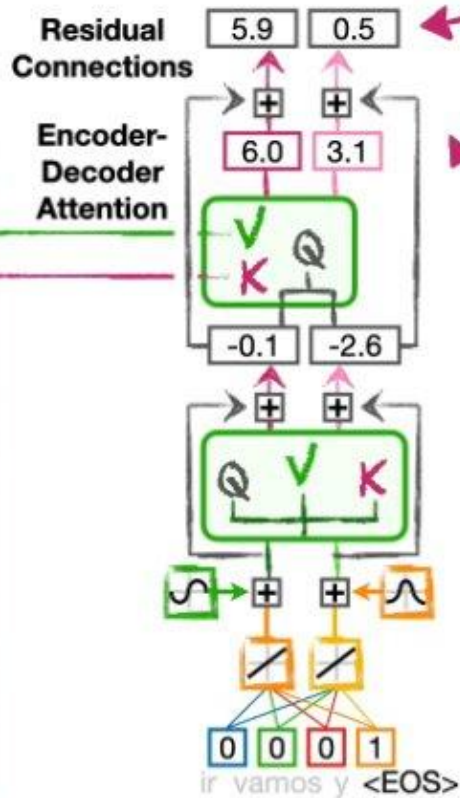
- **Challenges in Traditional NLP Models:**
 - **RNNs & LSTMs:** Sequential processing → slow, can't parallelize well
 - **Vanishing gradients:** Long-term dependencies are hard to learn
 - **Limited context window:** Only recent tokens matter the most
 - **High training time:** Especially with large datasets
-
- **The Breakthrough – Google's Transformer (2017):**
 - Introduced "**Attention is All You Need**"
 - Fully attention-based architecture (no RNNs)
 - Enables **parallel training**
 - Handles **long-range dependencies** efficiently



Encoder



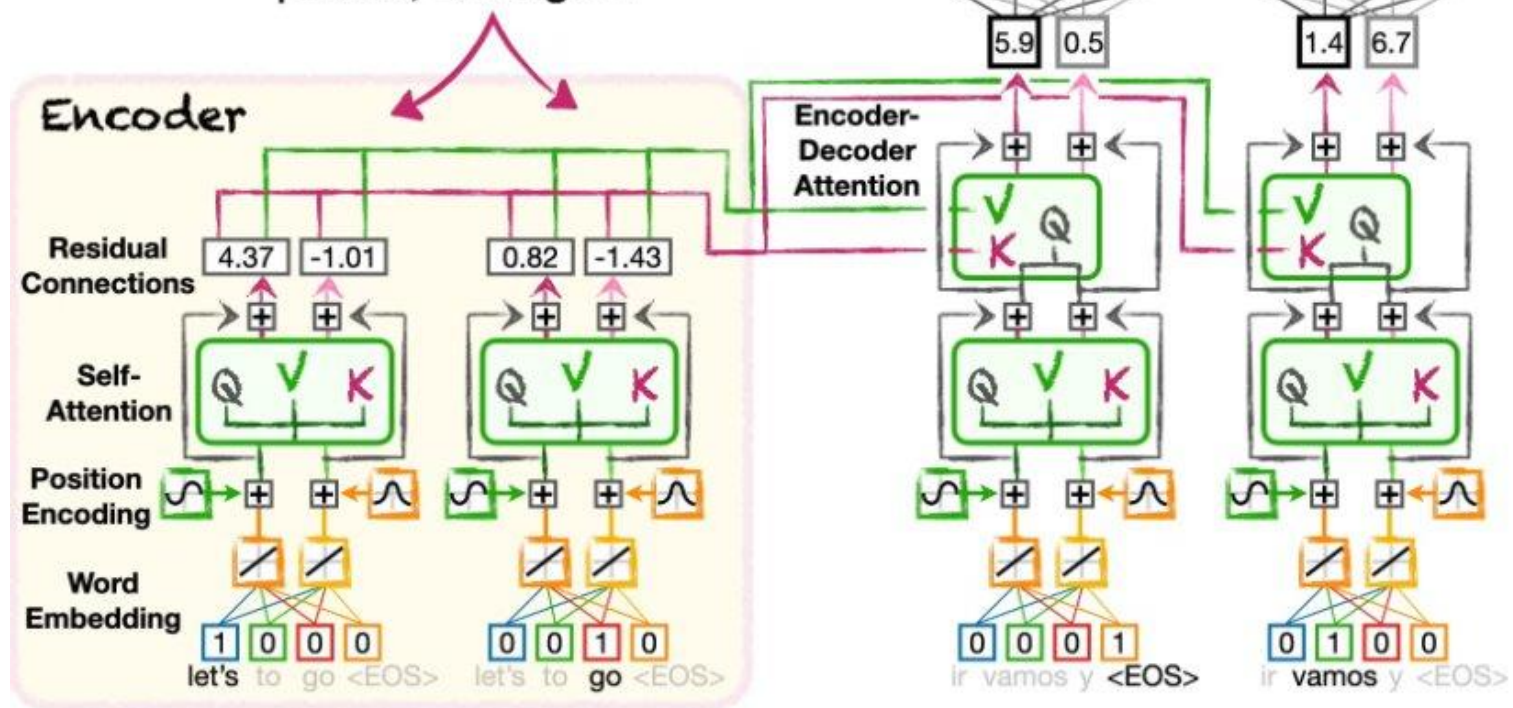
Now we add another set of
Residual Connections...



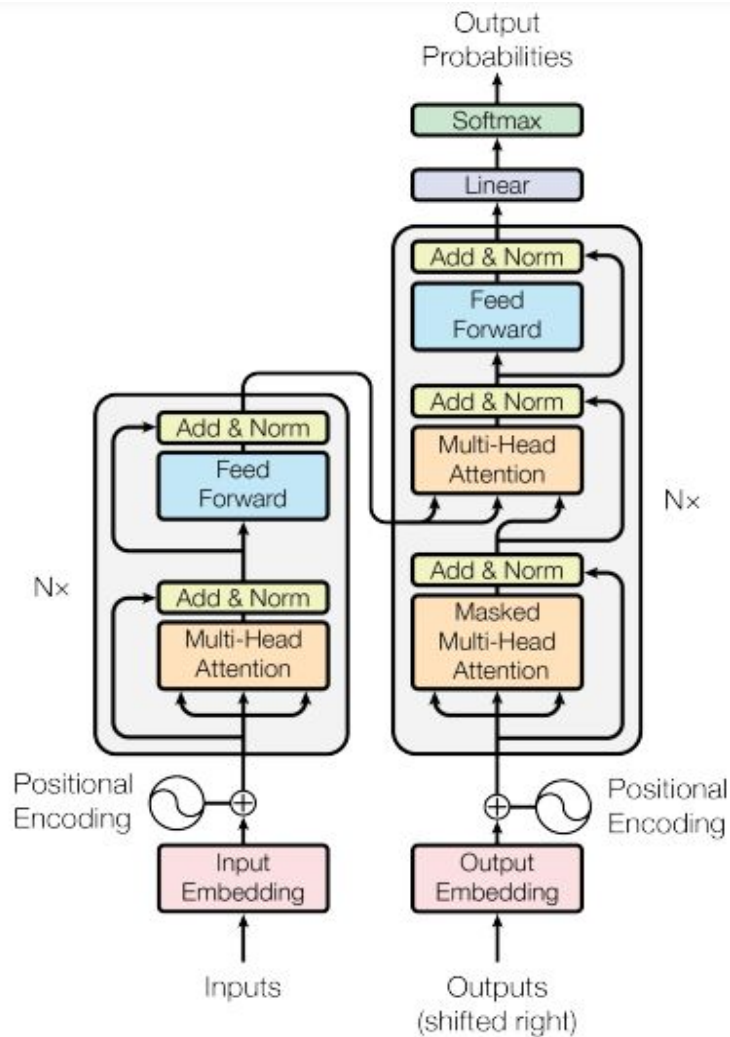
...that allow the
Encoder-Decoder Attention to focus on relationships between the output words and the input without having to preserve the **Self-Attention** or **Word and Position Encoding** that happened earlier.



At long last, we've shown
how a **Transformer** can
encode a simple input
phrase, **Let's go...**

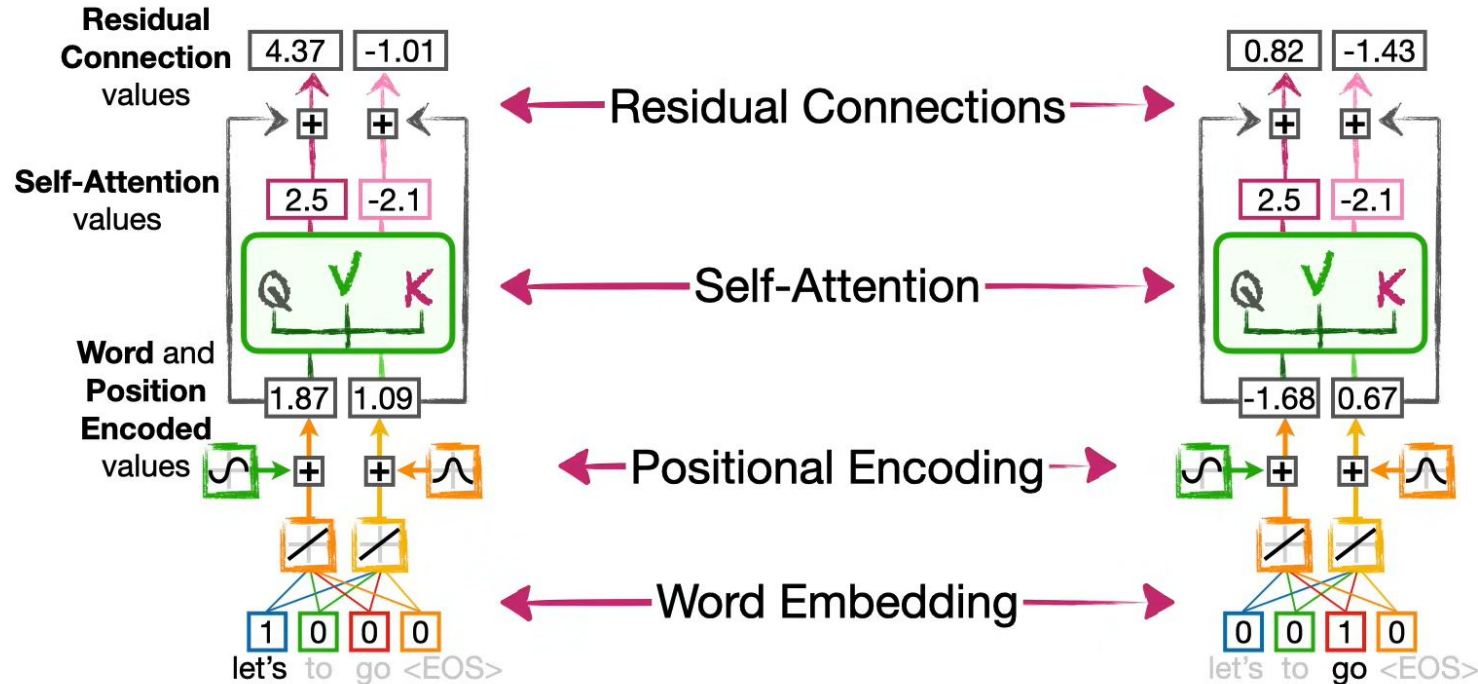


Transformers Architecture:





These **4** features allow the **Transformer** to:
encode words into numbers, encode the
positions of the words, encode the
relationships among the words, and relatively
easily and quickly train in parallel.



Word Embeddings

- Convert words into fixed-size dense vectors
- Capture semantic meaning (e.g., “king” - “man” + “woman” $\approx$ “queen”)
- Pretrained embeddings: Word2Vec, GloVe, or learned during training

Positional Encoding

- Transformers have no sense of word order
- Positional encoding adds order info to embeddings
- Uses sine/cosine functions to encode position into vectors

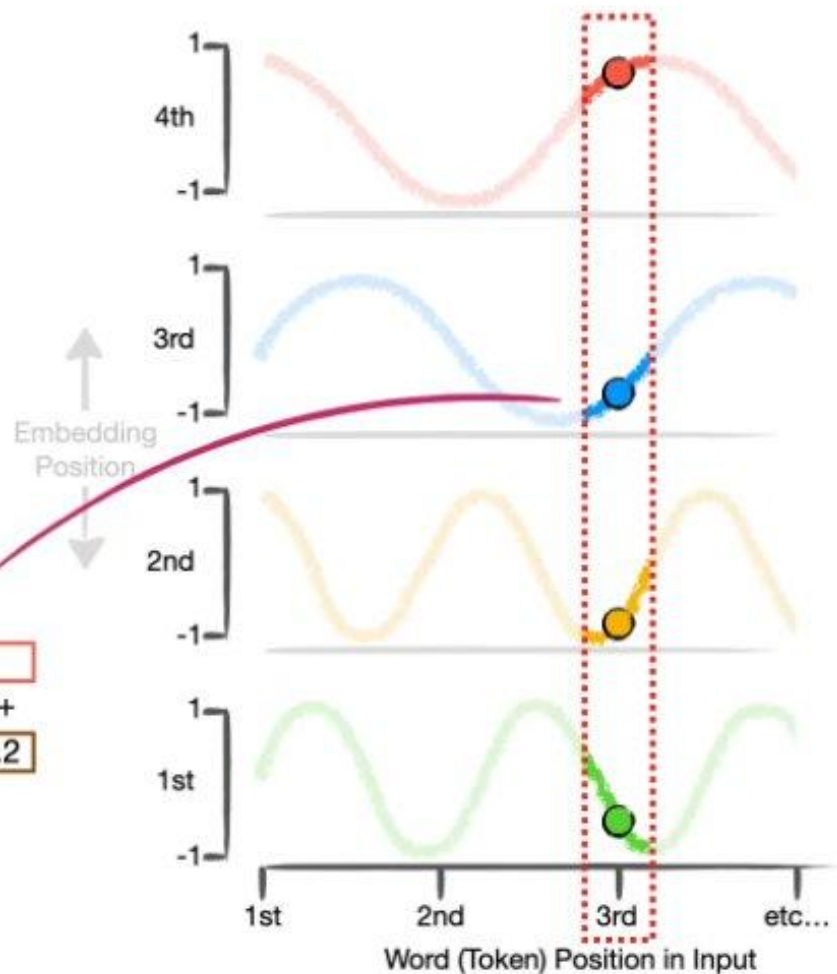
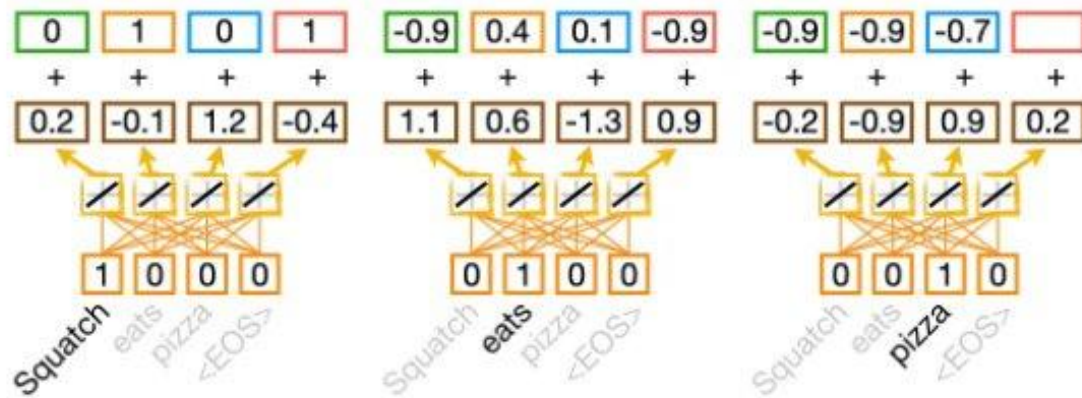
Simply,

word embedding = meaning

Positional Encoding = order



...we use the y-axis coordinates on the squiggles that correspond to the x-axis coordinate for the third word.



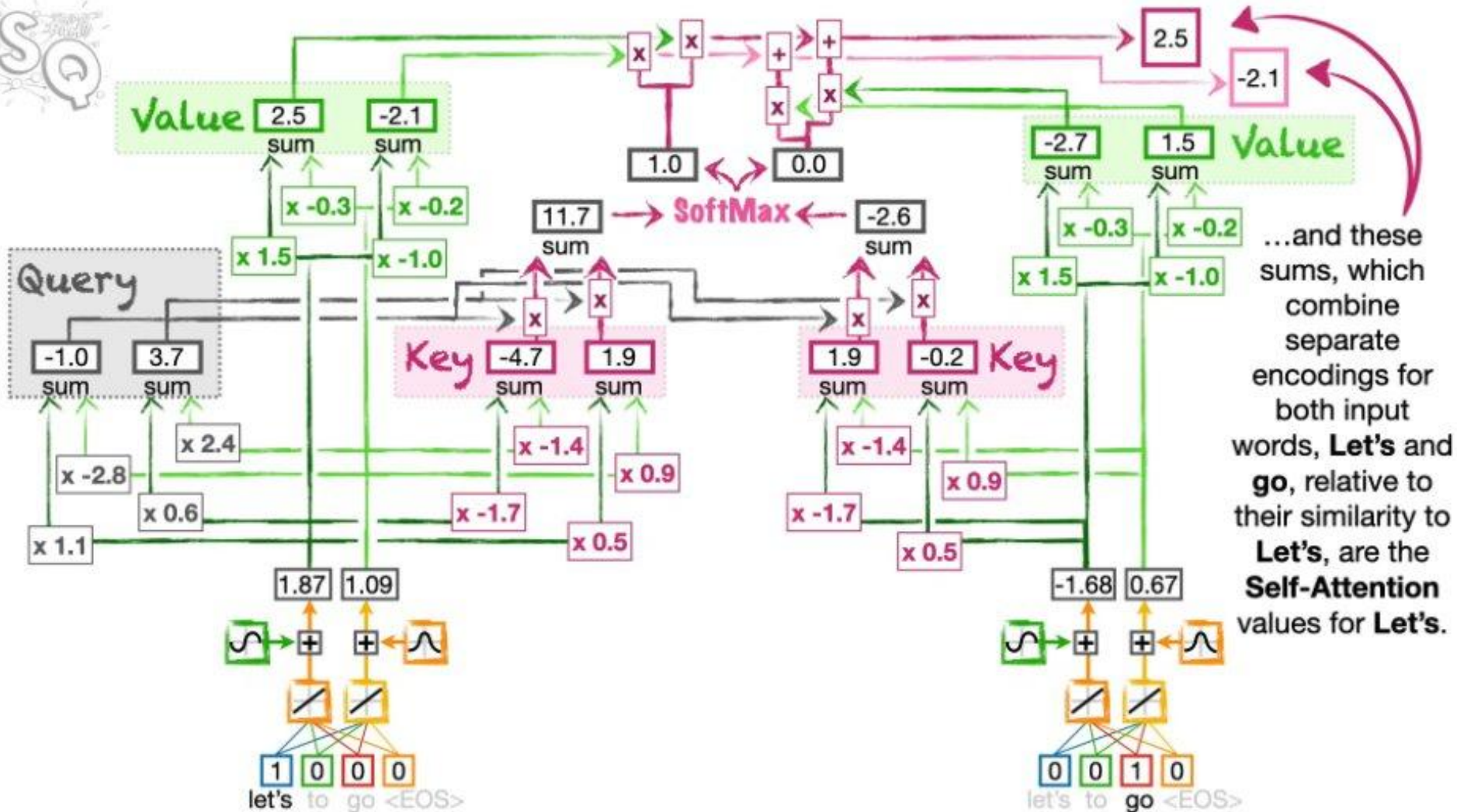
Self Attention:

Self-Attention

- Helps model focus on relevant words in a sentence
- Computes weighted importance of other words for each word
- Allows global context understanding

Query, Key, and Value (QKV)

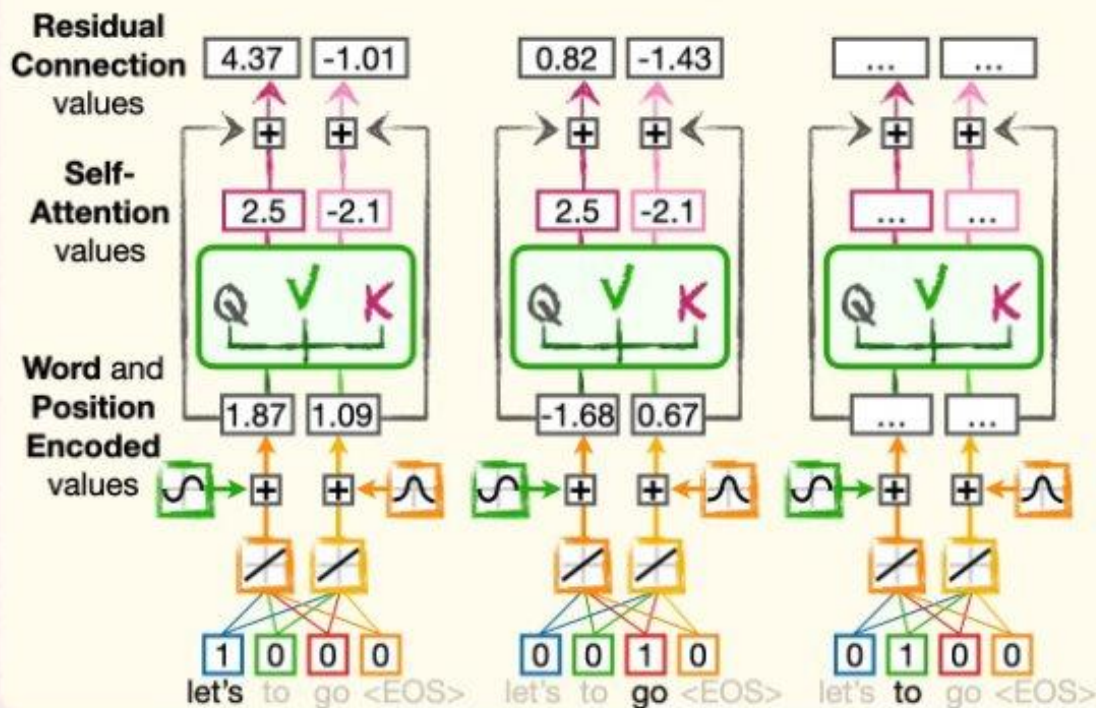
- Each word is transformed into 3 vectors: **Query (Q)**, **Key (K)**, **Value (V)**
- Attention Score = Similarity between Q and K
- Output = Weighted sum of V's based on attention scores





Doing all of the computations at the same time, rather than doing them sequentially for each word...

Encoder



...means we can process a lot of words relatively quickly on a chip with a lot of computing cores, like a **GPU** (graphics processing unit), or on multiple chips in the cloud.

Encoder-Decoder Attention

Where it fits:

- Used in the **decoder**, after masked self-attention

What it does:

- Allows the decoder to attend to **encoder outputs**
- Helps the decoder use the **input sentence context** while generating output

How it works:

- Decoder's Query + Encoder's Key & Value
- Helps align input tokens with output tokens (e.g., translation tasks)